Cuckoo search algorithm

```python
import numpy as np
import random
import math

# ------------------------------
# Step 1: Initialize Parameters
# ------------------------------

cities = {
    'A': (0, 0),
    'B': (2, 6),
    'C': (5, 4),
    'D': (6, 1),
    'E': (3, 2)
}

city_names = list(cities.keys())
num_cities = len(city_names)

n = 10               # Number of host nests
Pa = 0.25            # Discovery probability
MaxIter = 10        # Maximum iterations
alpha = 0.5          # Levy flight step factor


# ------------------------------
# Step 2: Calculate Distance
# ------------------------------

def calculate_distance(city1, city2):
    x1, y1 = cities[city1]
    x2, y2 = cities[city2]

    return math.sqrt((x2-x1)**2 + (y2-y1)**2)
```

```
# ------------------------------
# Step 3: Fitness Function
# ------------------------------

def fitness(route):
    total_distance = 0

    for i in range(num_cities - 1):
        total_distance += calculate_distance(
            route[i], route[i+1]
        )

    # Return to starting city
    total_distance += calculate_distance(
        route[-1], route[0]
    )

    return total_distance


# ------------------------------
# Step 4: Generate Initial Nests
# ------------------------------

def generate_random_tour():
    route = city_names.copy()
    random.shuffle(route)
    return route


nests = [generate_random_tour() for _ in range(n)]

fitness_values = [fitness(route) for route in nests]


# ------------------------------
```

```python
# ------------------------------
# Step 5: Levy Flight
# ------------------------------

def levy_flight(route):
    new_route = route.copy()

    # Levy-distributed step size
    u = random.gauss(0, 1)
    v = random.gauss(0, 1)

    beta = 1.5

    sigma = (
        math.gamma(1 + beta) *
        math.sin(math.pi * beta / 2) /
        (
            math.gamma((1 + beta) / 2) *
            beta * 2 ** ((beta - 1) / 2)
        )
    ) ** (1 / beta)

    step = u * sigma / (abs(v) ** (1 / beta) + 1e-10)

    # Convert continuous step into swap operations
    swaps = max(1, min(
        num_cities - 1,
        int(abs(alpha * step) * num_cities) + 1
    ))

    for _ in range(swaps):
        i, j = random.sample(range(num_cities), 2)
        new_route[i], new_route[j] = (
            new_route[j], new_route[i]
        )

    return new_route
```

```
# ------------------------------
# Step 6: Cuckoo Search Algorithm
# ------------------------------

best_index = np.argmin(fitness_values)
best_route = nests[best_index].copy()
best_distance = fitness_values[best_index]

for iteration in range(MaxIter):

    # Generate new solutions using Levy Flight
    for i in range(n):

        new_route = levy_flight(nests[i])
        new_fitness = fitness(new_route)

        # Select a random host nest
        j = random.randint(0, n - 1)

        # Compare cuckoo egg with host egg
        if new_fitness < fitness_values[j]:

            nests[j] = new_route
            fitness_values[j] = new_fitness

    # ------------------------------
    # Step 7: Abandon Worst Nests
    # ------------------------------

    for i in range(n):

        if random.random() < Pa:

            nests[i] = generate_random_tour()
            fitness_values[i] = fitness(nests[i])
```

```
    # ------------------------------
    # Step 7: Abandon Worst Nests
    # ------------------------------

    for i in range(n):

        if random.random() < Pa:

            nests[i] = generate_random_tour()
            fitness_values[i] = fitness(nests[i])

    # ------------------------------
    # Step 8: Update Best Solution
    # ------------------------------

    current_best = np.argmin(fitness_values)

    if fitness_values[current_best] < best_distance:

        best_distance = fitness_values[current_best]
        best_route = nests[current_best].copy()

    print(
        "Iteration:", iteration + 1,
        "Best Distance:", round(best_distance, 2)
    )


# ------------------------------
# Step 9: Display Final Result
# ------------------------------

print("\n------ FINAL RESULT ------")

print("Best Route:",
      " -> ".join(best_route + [best_route[0]]))
```

```
            best_route = nests[current_best].copy()

        print(
            "Iteration:", iteration + 1,
            "Best Distance:", round(best_distance, 2)
        )


    # ------------------------------
    # Step 9: Display Final Result
    # ------------------------------

    print("\n------ FINAL RESULT ------")

    print("Best Route:",
          " -> ".join(best_route + [best_route[0]]))

    print("Minimum Travel Distance:",
          round(best_distance, 2))
```

```
Iteration: 1 Best Distance: 19.86
Iteration: 2 Best Distance: 19.86
Iteration: 3 Best Distance: 19.86
Iteration: 4 Best Distance: 19.86
Iteration: 5 Best Distance: 19.86
Iteration: 6 Best Distance: 19.86
Iteration: 7 Best Distance: 19.86
Iteration: 8 Best Distance: 19.86
Iteration: 9 Best Distance: 19.86
Iteration: 10 Best Distance: 19.86

------ FINAL RESULT ------
Best Route: D -> E -> A -> B -> C -> D
Minimum Travel Distance: 19.86
```